

La tienda gana direcciones

AbortController y el skeleton cierran la API. React Router: cada pantalla con su URL. Y nace el checkout: el formulario controlado.

4 h 15 · L-M-V

Retomamos donde cortamos

STEERCL

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura + 3 correcciones **18'**
el punto de corte · bugs de la revisión del repo, en vivo

A. Remate de la clase 4 **0-90'**
solo lo pendiente · guion de la clase 4

B. Remate de la API **30'**
AbortController · el skeleton de carga

C. React Router **70'**
páginas · detalle por URL · 404 · Layout

D. El checkout nace **55'**
formulario controlado · validación por campo

F. Cierre + 5 tareas **20'**
push demostrado, dudas y anticipo

 **Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día** · se corta a las 3:50 donde toque

LA RESPUESTA A LA TAREA DIFÍCIL

¿Qué limpia la limpieza de un fetch? **Lo cancela**

`new AbortController()`

el controlador, en el efecto



`signal`

viaja como opción del fetch



`abort()`

la limpieza cancela el viaje

```
// Network, con StrictMode: monta y pide · desmonta y CANCELA · monta, pide y llega  
GET /products (canceled) → GET /products 200
```

El detalle fino: `AbortError` NO es un fallo. Es tu orden cumplida. El catch lo reconoce y lo deja pasar: ni estado de error ni respaldo — porque no pasó nada malo.

DE PÁGINA ÚNICA A APLICACIÓN

Una SPA con direcciones

`tienda.com/`

el catálogo

`tienda.com/producto/7`

el detalle — un enlace que puedes compartir

`tienda.com/checkout`

el pago

`tienda.com/loquesea`

nuestra 404, no una pantalla en blanco

Un solo HTML, muchas direcciones, **sin recargar jamás** — y el botón atrás por fin sabe de nosotros.

La pieza: `npm install react-router-dom` — la primera instalación desde la clase 1.

SEIS PIEZAS, UN MAPA

Las piezas del Router

BrowserRouter

envuelve todo en `main.tsx`: el que escucha la barra

Routes / Route

el mapa: qué dirección muestra qué página

Link

el enlace que **no recarga**: cambia la URL y la pantalla

useParams

la parte variable de la URL (`:id`) — llega como **string**

Outlet

"aquí va la página que toque": el Layout se escribe UNA vez

path="*"

el comodín: atrapa toda dirección desconocida → la 404

La arquitectura nueva: `components/` = piezas reutilizables · `pages/` = pantallas, una por dirección · `App` = el mapa. El estado se queda en `App`: las páginas van y vienen, **el carrito permanece**.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes del Router

1 · <a href> en vez de Link

funciona... recargando la página entera: el estado muere, el carrito parpadea, la SPA deja de ser SPA

2 · olvidar BrowserRouter

todo lo del Router revienta al arrancar — y el mensaje de la consola te lo dice con nombre y apellido

3 · comparar useParams sin convertir

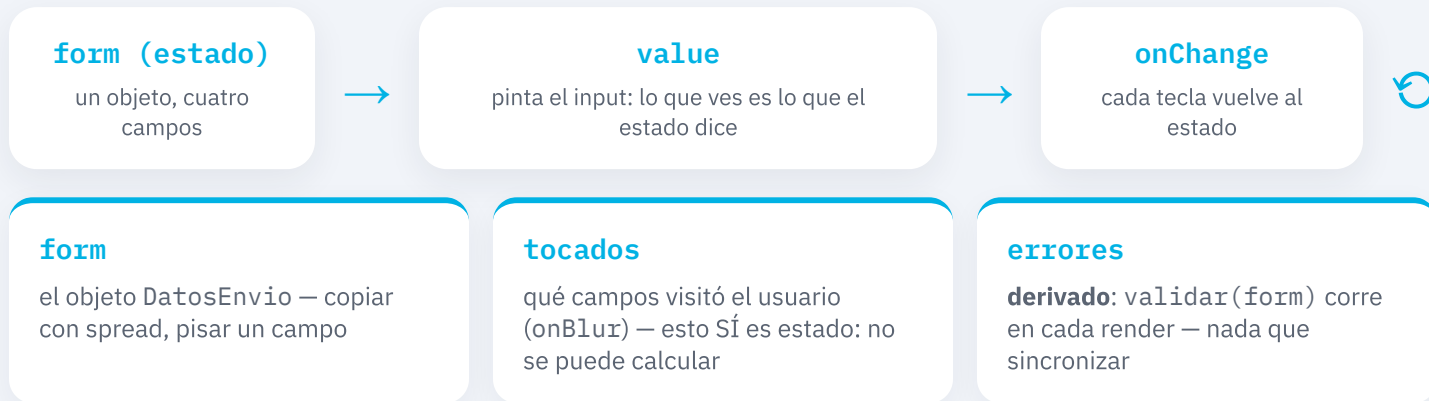
el id llega como string: '7' === 7 es falso → `Number(id)` antes del `find`

4 · F5 en producción da 404

en dev Vite resuelve las rutas; el servidor real necesita el `rewrite` de SPA — lo haremos al desplegar

EL CHECKOUT NACE

El formulario controlado



Ese trío — form, tocados, errores derivados — es EL patrón de formularios de React.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de formularios

1 · value sin onChange

el input queda CONGELADO: React manda y nadie actualiza el estado — la consola además lo advierte

2 · errores en useState + efecto

el anti-efecto de la clase pasada, versión formularios: los errores se **derivan**, no se sincronizan

3 · validar solo con required / minLength

los atributos nativos son una red, pero el usuario merece mensajes tuyos, por campo

4 · olvidar preventDefault

la página recarga, el estado muere y el pedido nunca existió

HASTA DONDE LLEGAMOS HOY

Lo de hoy

cancelar

la limpieza de un fetch lo cancela:
controlador → signal → abort().
AbortError no es fallo

skeleton

la espera con forma — y la única
excepción legal de la key por índice

SPA con direcciones

BrowserRouter · el mapa de Routes ·
Link que no recarga

useParams + 404

la parte variable llega como string;
path="*" atrapa lo desconocido

Layout + Outlet

el marco se escribe una vez; toda
página vive adentro

formulario controlado

un objeto + tocados; los errores son
un derivado, no un estado

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos, con la grabación al lado. Es LA tarea.

INTERMEDIO

Tu skeleton fiel: mismos bloques que tu tarjeta real + `motion-reduce:animate-none`.

INTERMEDIO

El detalle completo: tu campo extra de la API en la página de detalle + el título de la pestaña con el nombre del producto.

DIFÍCIL

Mis pedidos: la página /pedidos que lee lo guardado por el checkout (con aduana) y lista fecha, items y total — con su estado vacío.

DIFÍCIL

El checkout protegido a medias: con carrito vacío, mensaje con enlace al catálogo en vez del formulario. ¿Ruta, condicional o ambos? Justifica en un comentario.

Próxima clase — retomamos donde cortamos

¿Cuántas manos tocan el carrito **antes de llegar al botón?**

App → Layout → Header → ...

App → Home → ProductCard → ... ← el carrito viaja por props a TODAS partes

// esa incomodidad tiene nombre y solución: **Context**

El estado global de verdad, con nuestros propios hooks (**useCart**). Y con Context en la mano, el proyecto gana su arquitectura: **refactor guiado al estilo hexagonal** — el mismo que vale un punto extra en el proyecto final.

Trae las tareas hechas y el build en cero. Nos vemos.